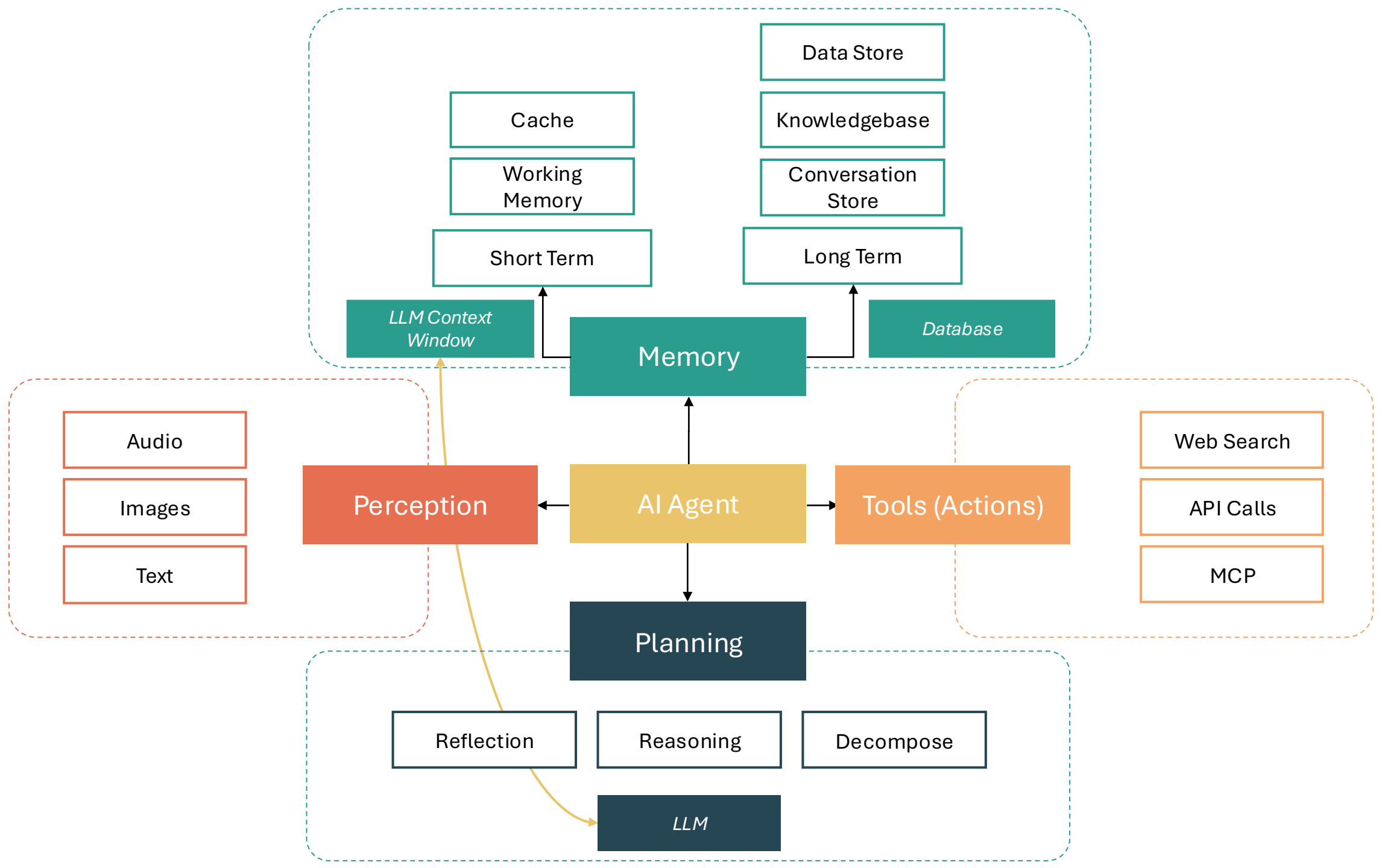




ما هو الفرق بين الوكيل Agent والـ Assistant والـ Chatbot؟

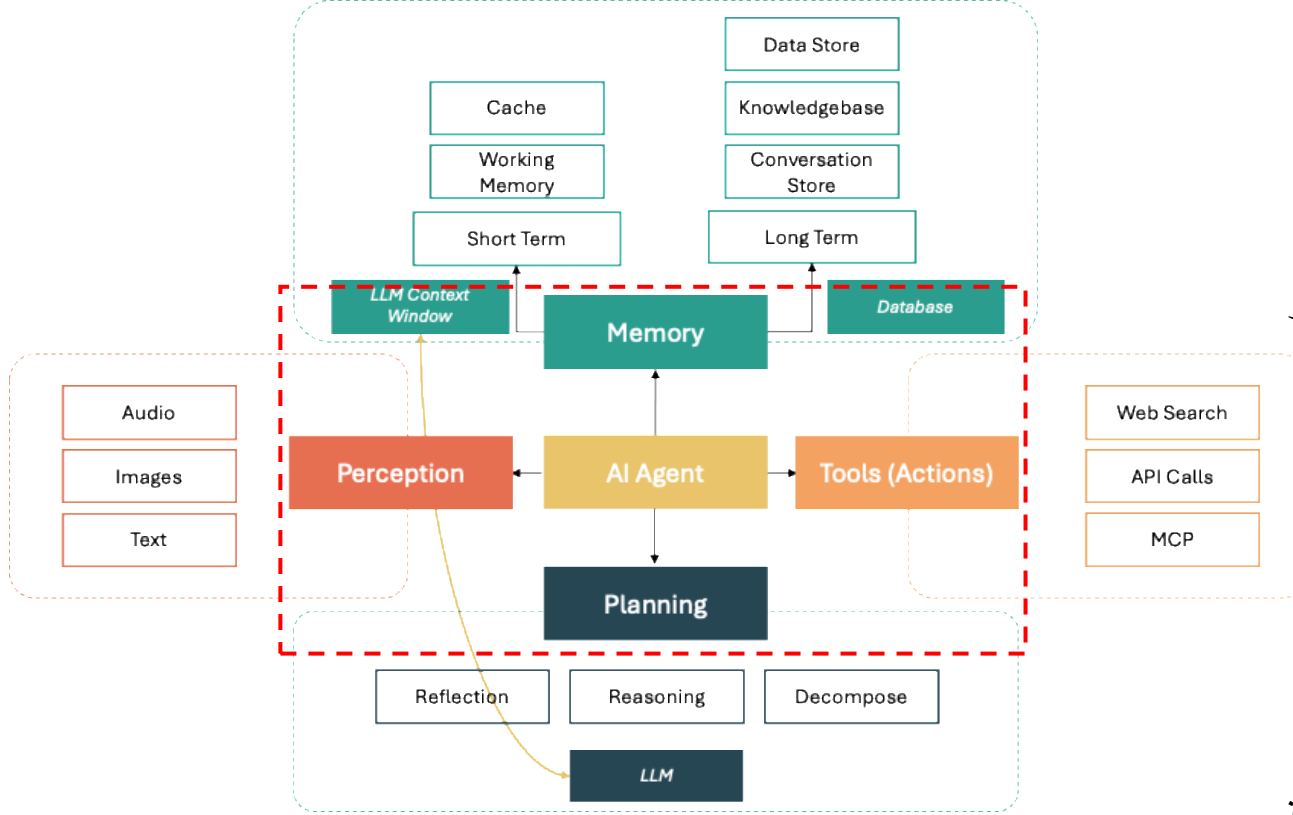
يكثر الحديث عن الفروقات ولكن أبسطها هو أن:

- Chatbot يساعدك في Tell me how to do it
- Assistant يساعدك في Help me do it
- Agent يساعدك في Do it for me

الوكيل كيان مستقل قادر على التخطيط والتذكر، بينما الـ Chatbot قادر على الإجابة فقط عن أسئلة محددة.

في الصورة على اليسار، نجد العناصر المشكلة للوكيل:

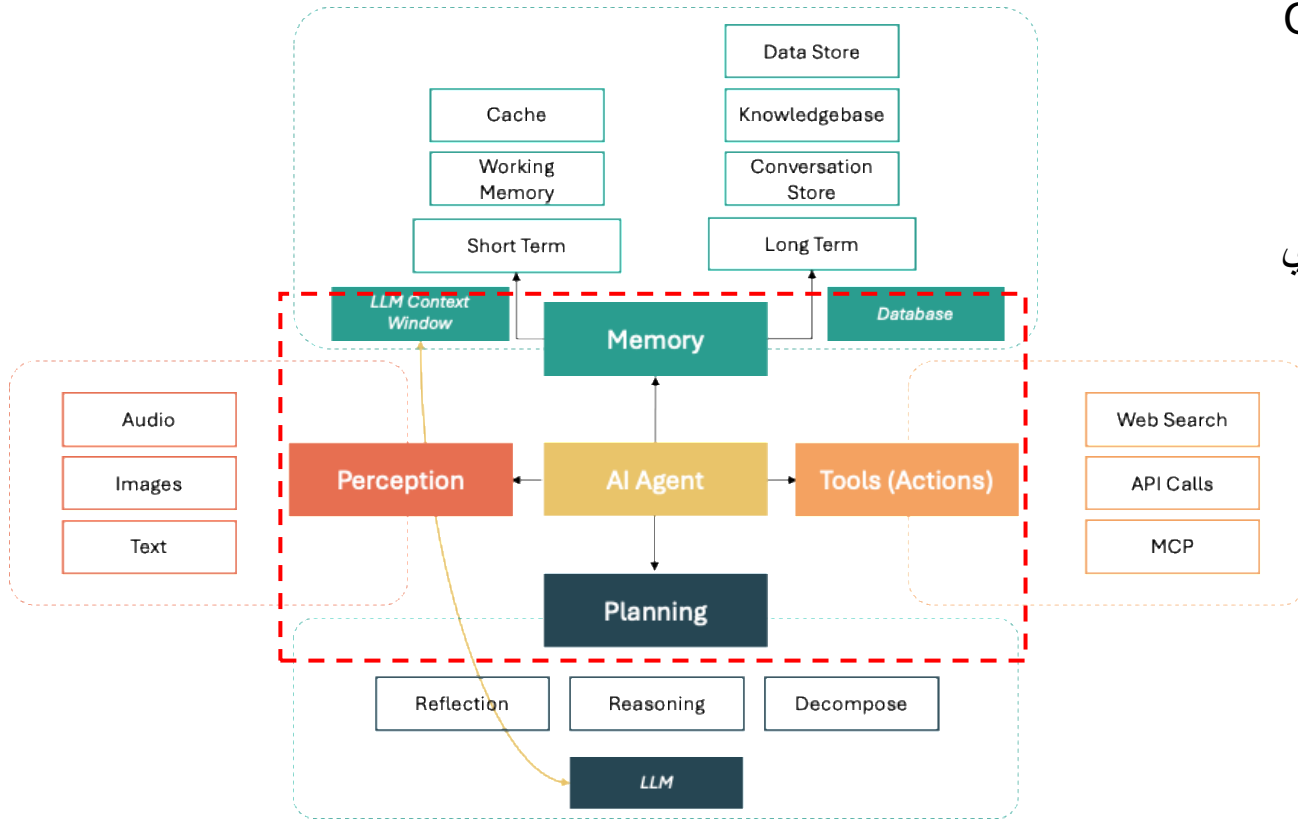
- قدرة على التخطيط والاستنتاج Planning and Reasoning
- لتفكيك طلب معقد لخطوات بسيطة باستخدام نماذج لغوية
- الاستقلالية Autonomy والعمل في دورات Loops حتى يتحقق الهدف من كل مهمة
- استخدام الأدوات Tools والبحث وتنفيذ المهام
- الذاكرة قصيرة المدى (تذكر ما تم عمله في الخطوة الأولى أثناء تنفيذ الخامسة) وطويلة المدى (تذكر أن طريقة تنفيذ المهمة السابقة لم تعجبك)
- القدرة على التعامل مع أشكال متنوعة من البيانات



ما هو الفرق بين الوكيل Agent والـ Assistant والـ Chatbot؟

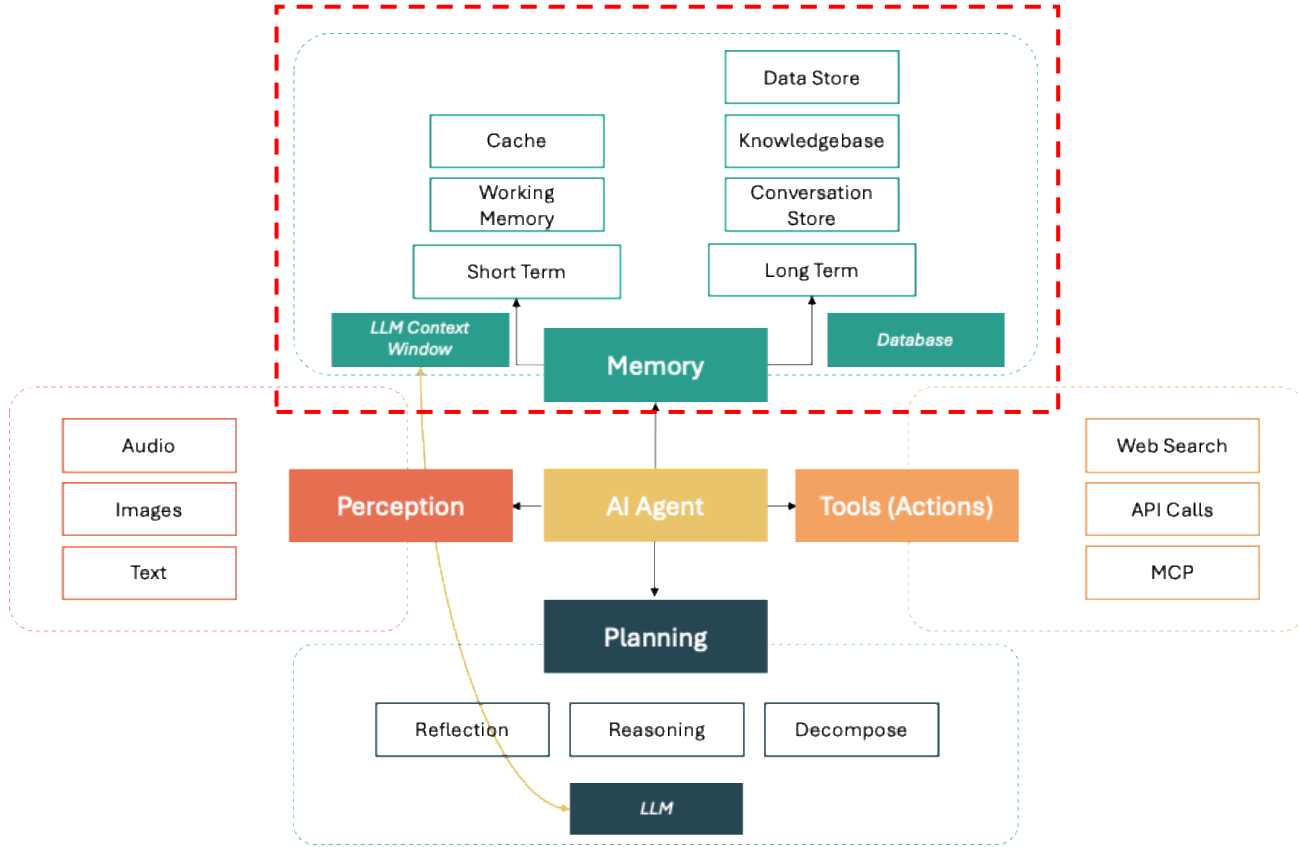
طبعاً توجد تداخلات وتنويعات بين هذه الثلاث أنواع مثلاً تجد Chatbot مزود بذاكرة قصيرة: Stateful Chatbot

أو وكيل يستخدم الـ LLM فقط لفهم طلب المستخدم Intent Resolver ومن ثم تنفيذ Workflow محدد ومبرمج مسبقاً (أي لا يُترك للوكيل استخدام الأدوات وتخطيط المهام بحرية مطلقة)



هل يتعين على الوكيل استدعاء الـ LLM مع كل سؤال يسأله المستخدم؟

الإجابة لا، لأن كل عملية استدعاء تستغرق وقتاً ولها كلفة. يضاف على ذلك أن الـ Tokens الممثلة لكلمة واحدة باللغة العربية أضعاف الإنجليزية (في الإنجليزية، $1.3 \text{ tok} \sim 1 \text{ w}$ ، بينما في العربية $1 \text{ w} \sim 4 \text{ tok}$) مما يملأ الـ Context Window ويزيد الكلفة والتأخير Latency.

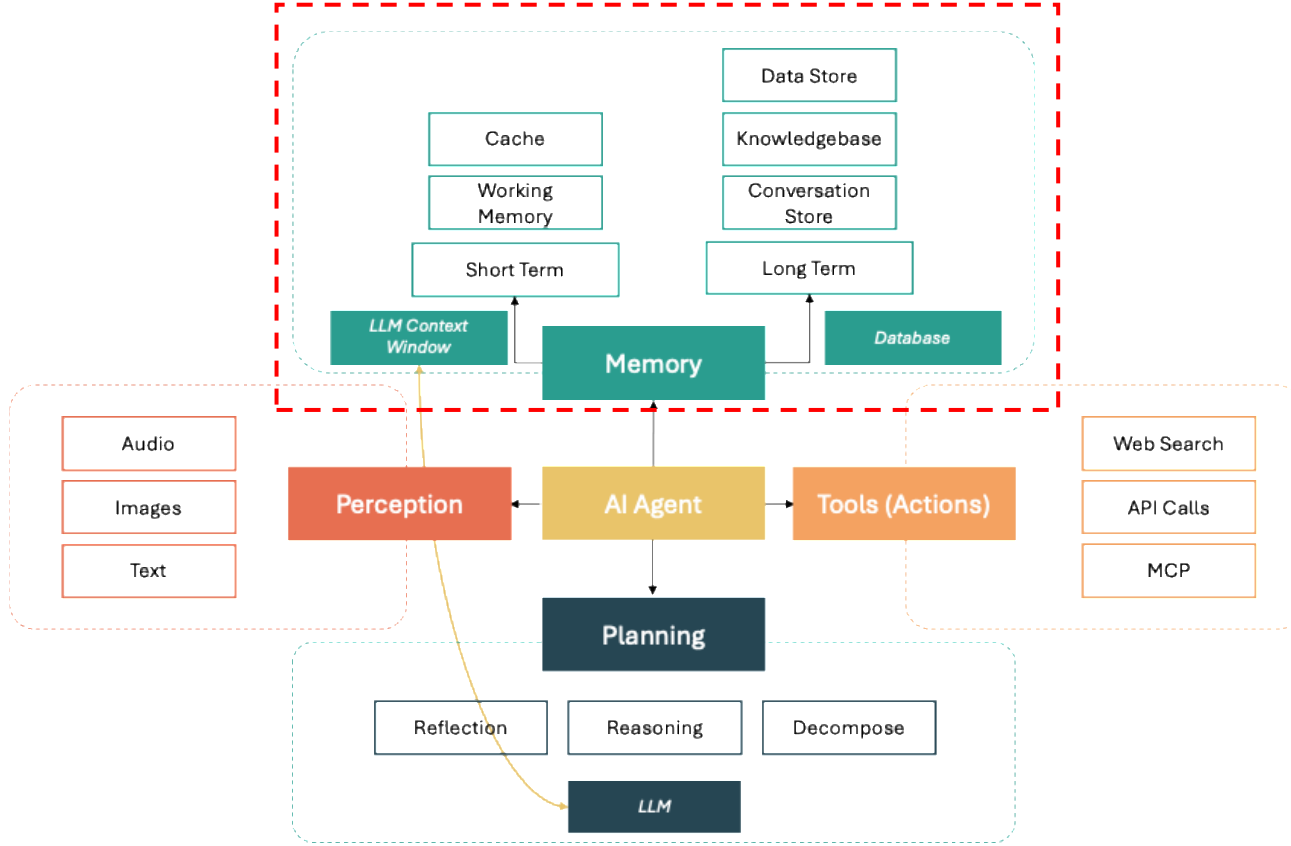


يمكن استخدام الـ Memory في ذلك. ففي حال سأل المستخدم السؤال مرتين، وإن بصيغة مختلفة، يمكن استدعاء الإجابة من الذاكرة قصيرة المدى (Semantic Caching) أو طويلة المدى كما في الصورة. ينطبق الأمر على الأسئلة الأكثر شيوعاً، حيث يمكن توفير الوقت والكلفة باستحضار الإجابة من ذاكرة الوكيل، أو الذاكرة المشتركة Shared Memory للوكلاء العاملين الذين يخدمون مستخدمين آخرين.

هل يتعين على الوكيل استدعاء الـ LLM مع كل سؤال يسأله المستخدم؟ إدارة ذاكرة الوكيل

تقوم أنظمة إدارة الذاكرة Memory Systems مثل Mem0 بإدارة وتنظيم ذاكرة الوكيل الذكي. فمثلاً تقوم بعملية Token Compression لتخزين المعلومات بعد تلخيصها Summarization وحذف الـ Tokens المكررة ومنخفضة القيمة. ولكن يجب الحذر من الـ Semantic Compression Loss وهو فقدان القيمة الدلالية والمعلومات المفتاحية عبر الإفراط في حذف الـ Tokens.

أغلب أنظمة إدارة الذاكرة المتاحة غير مهيأة للتعامل الأمثل مع ارتجاع النصوص العربية Retrieval، ما يؤدي لنتائج خاطئة في بعض الأحيان (لذلك نشر المهندس هشام هارون مشروع "ذاكرة Dhakira" لحل هذا التحدي).



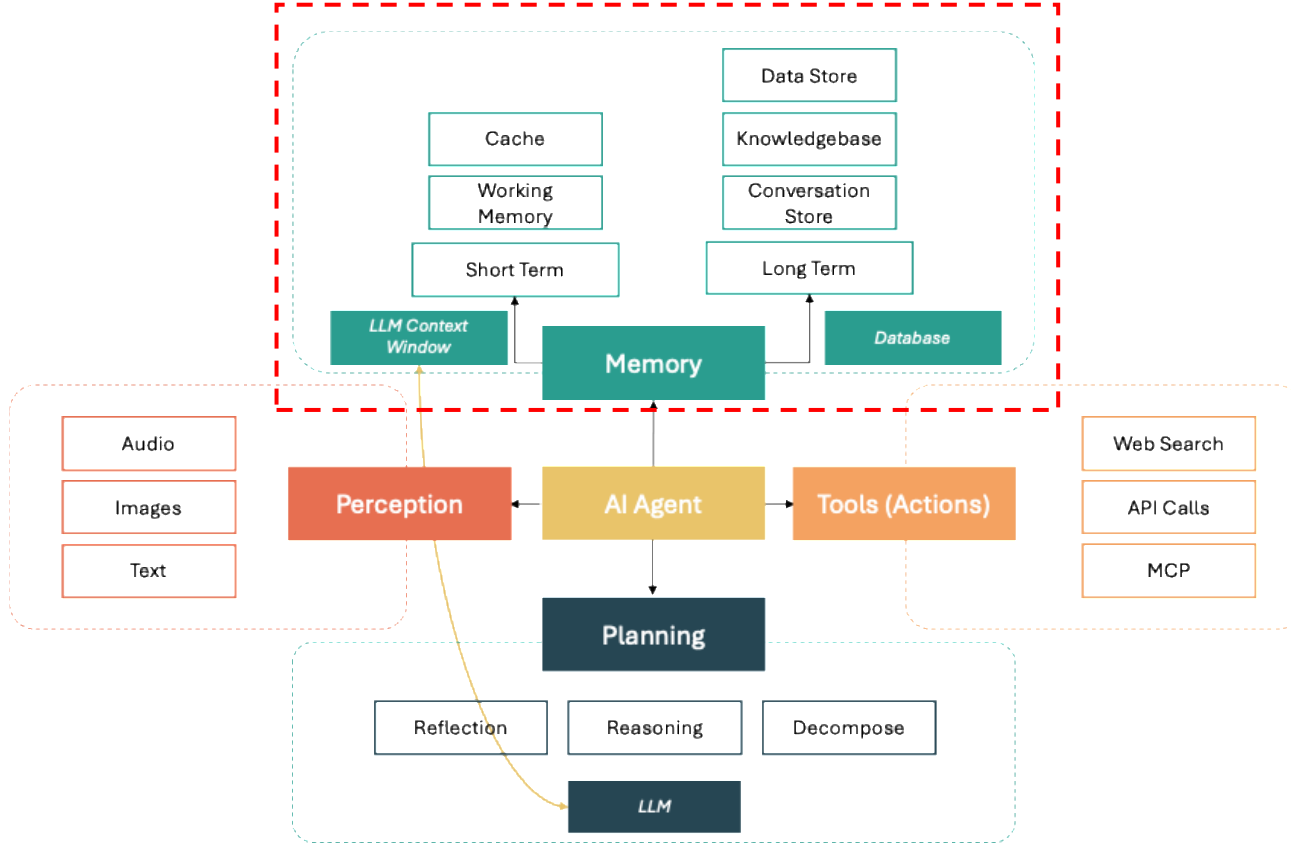
هل يتعين على الوكيل استدعاء الـ LLM مع كل سؤال يسأله المستخدم؟ إدارة ذاكرة الوكيل

طريقة البحث الموصى بها في مشروع "ذاكرة" لارتجاع البيانات هي الأسلوب الهجين الذي يسترجع قيم عبر ٣ أنماط بحثية:

- Vector Search – للبحث الدلالي Semantic
- خوارزمية BM25 – للبحث في المفردات Lexical (يستخدم TF-IDF وهي الخوارزمية الافتراضية في Elastic and Lucene)
- Graph Search – للبحث في السياق Contextual والعلاقات

يتم ترتيب القيم المرتجعة عبر آلية RRF Reciprocal Rank Fusion وهي آلية Unsupervised تدمج القيم المرتجعة من طرق بحث مختلفة في قائمة واحدة.

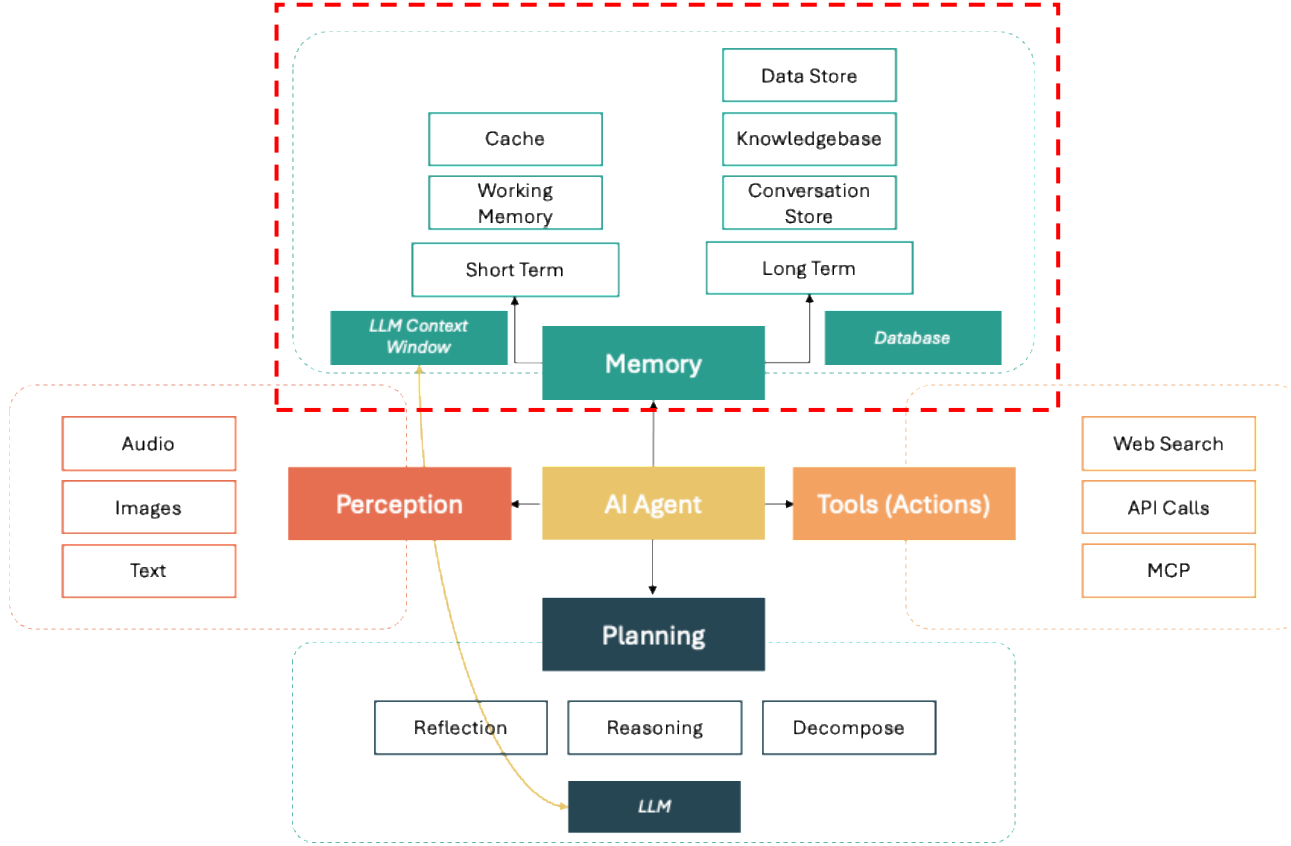
هناك تنويعات أخرى من RRF مثل Weighted RRF و Temporal/Recency Boosted RRF.



هل يتعين على الوكيل استدعاء الـ LLM مع كل سؤال يسأله المستخدم؟ إدارة ذاكرة الوكيل

من المهم التنبيه إلى أن آلية RRF في الترتيب Ranking تختلف عن نماذج الـ Reranking، مثل Cohere Rerank and BGE Reranker في أن الأولى تركز على ترتيب النتائج فقط، بينما النماذج الأخرى تستخدم خوارزميات الشبكات العصبية لقياس التشابه بين جملة البحث والنتائج المرتجعة).

لقياس مدى فعالية نظام الذاكرة بعد تطبيق آلية البحث الثلاثية، يمكن مقارنة نتائج البحث لذات الأسئلة في الوضع التقليدي وبعد التحسين وحساب الـ F1 Score وحسابات الـ Precision and Recall.



هل يتعين على الوكيل استدعاء الـ LLM عبر الـ API دائما؟ أم استخدام الـ Self-hosted؟

النماذج المستضافة خارجيا متفوقة في الغالب ولكنها مكلفة ماديا وزمنيا.

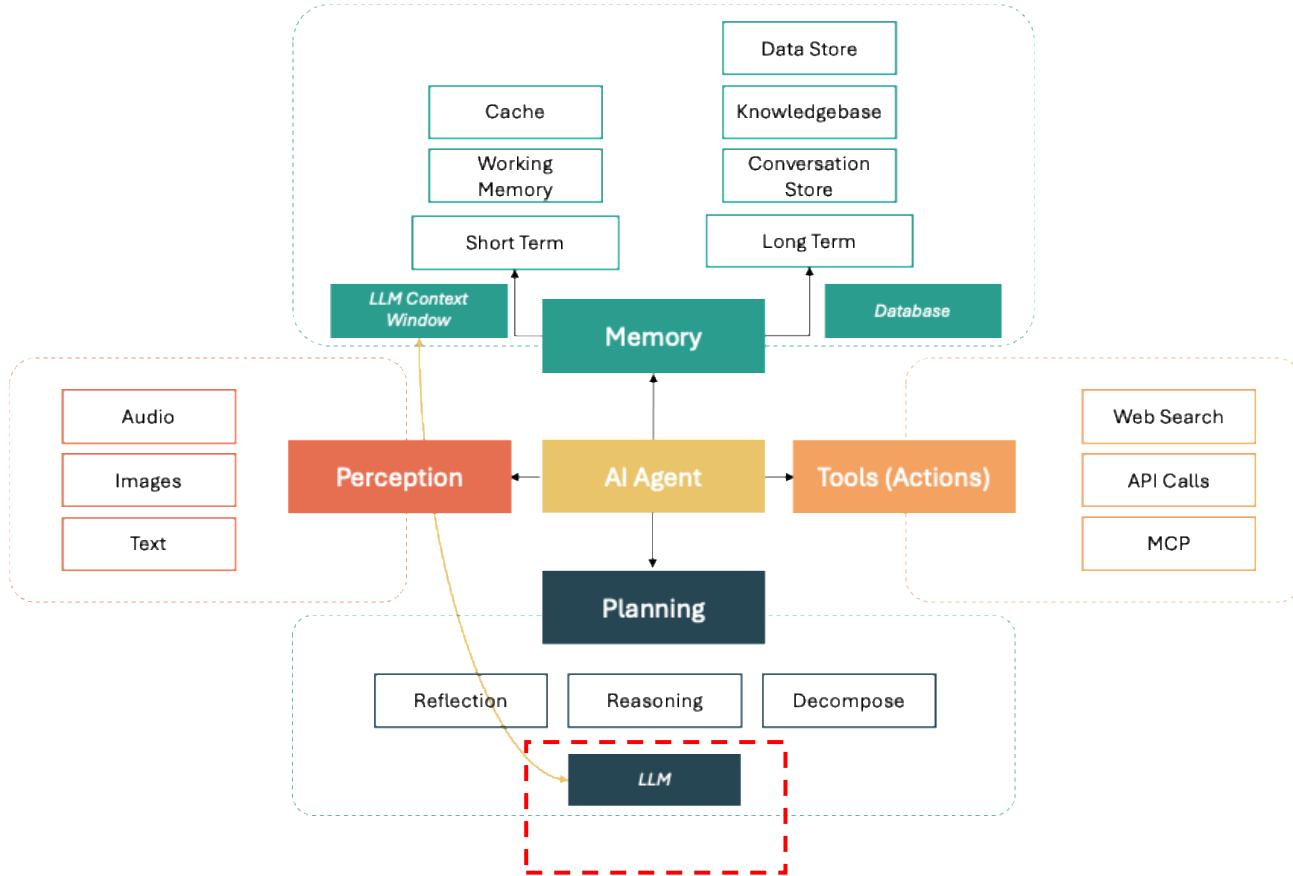
مثلا تكلفة المليون Tokens في كلود:

Model	Input	Output
Claude 4.6/4.5 Sonnet	\$3-\$6	\$15-\$22.50
Claude 4.6/4.5 Opus	\$5-\$10	\$25-\$37.50

وهي تكاليف مرتفعة جدا مقارنة بما ستدفعه على نموذج مستضاف محليا لديك.

بينما التكلفة الزمنية، على مقياسي TTFT (أي Time To First Token وهو الذي يقيس مدة الانتظار لاستقبال أول رد) ومقياس TPOT (أي Time Per Output Token الذي يقيس زمن إظهار الكلمات)

Metric / Mode	API	Self-hosted
TTFT	200-800 ms	30-150 ms
TPOT	80-120 tok/s	30-70 tok/s



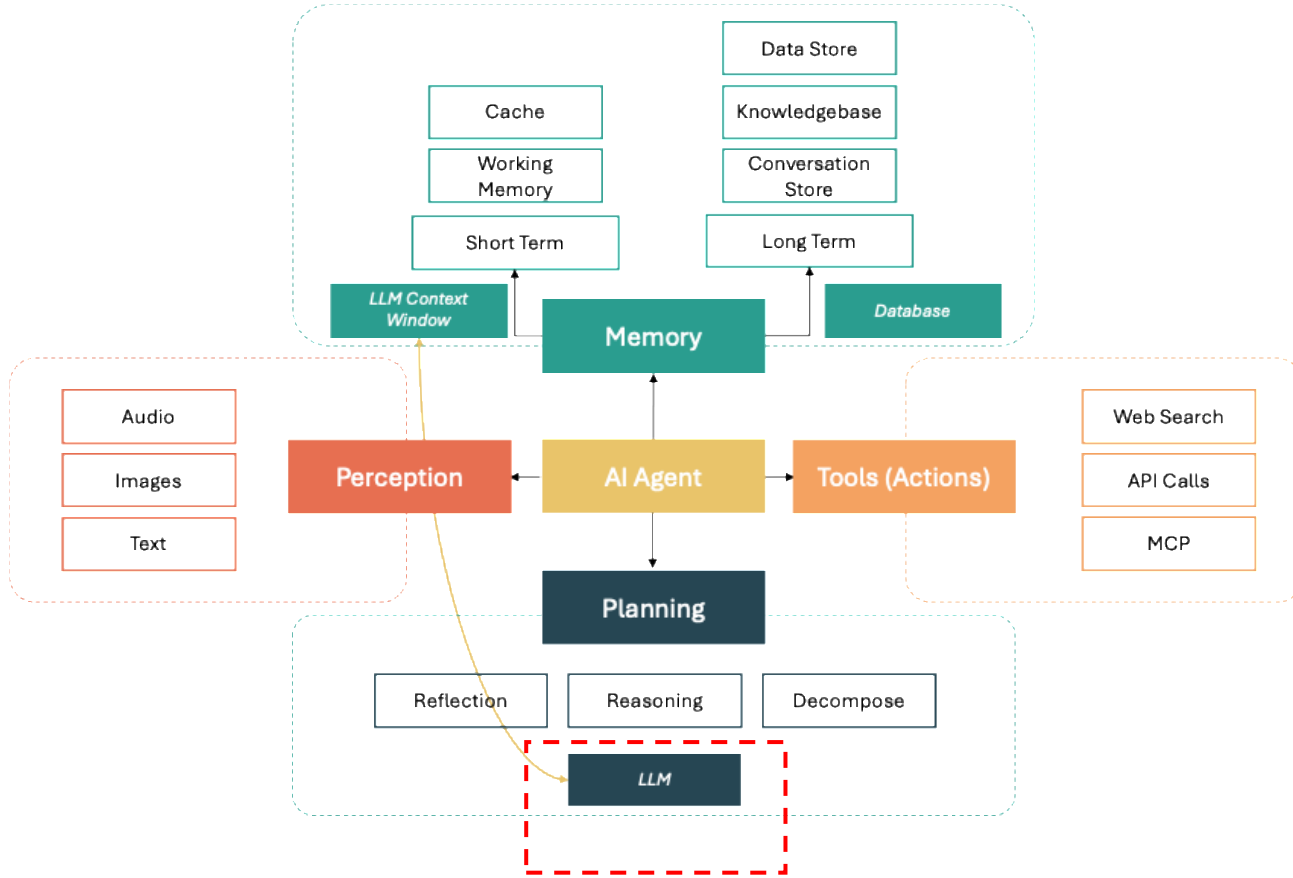
هل يتعين على الوكيل استدعاء الـ LLM عبر الـ API دائما؟ أم استخدام الـ Self-hosted؟

يتوجب التعامل من التذبذب Jitter الممكن حصوله من النماذج المستضافة محليا وخارجيا مثل:

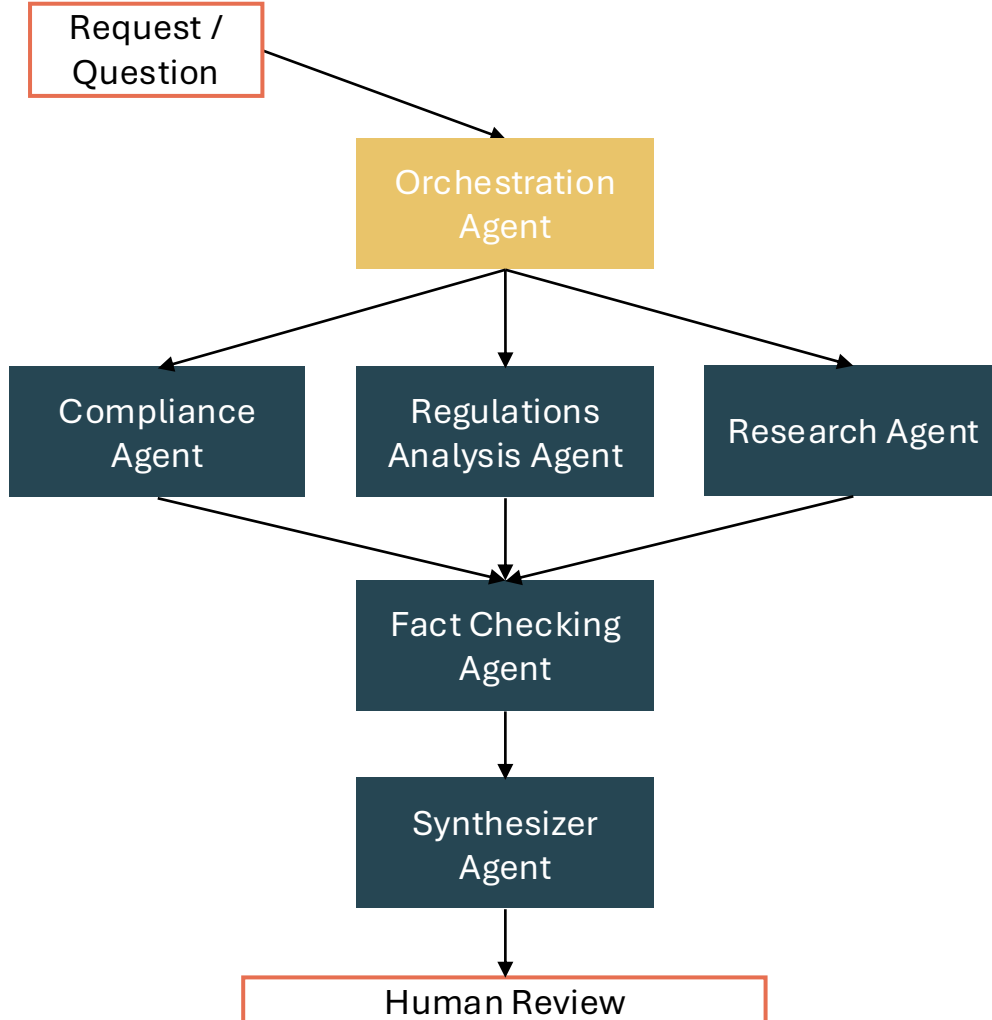
- Inference Latency Jitter (Performance Inconsistency)
- Data and Input Jitter (Multimodal/Inertial Data)
- Output/Generation Jitter (Response Variability)
- Retry and Network Jitter

التوجيه المناسب هو استخدام آلية هجينة تحلل السؤال Query Analyzer، وتستخدم بناء على ذلك:

- نموذج محلي للأسئلة البسيطة Simple Queries
- نموذج خارجي للأسئلة المعقدة Complex Reasoning



هل أكتفي بوكيل واحد للبحث في الاقتصاد السياسي وتشريعاته أم ألجأ لنظام متعدد الوكلاء (أي وكيل للبحث، وكيل للتشريعات، وكيل للتحقق من النتائج...)?



في الأنظمة متعددة الوكلاء Multi-Agent، قد يكون الزمن المستغرق في تنسيق المهام فيما بين الوكلاء طويلا جدا. كما أن الأخطاء في الإجابات والمعالجة تتراكم وتتضاعف من وكيل لوكيل.

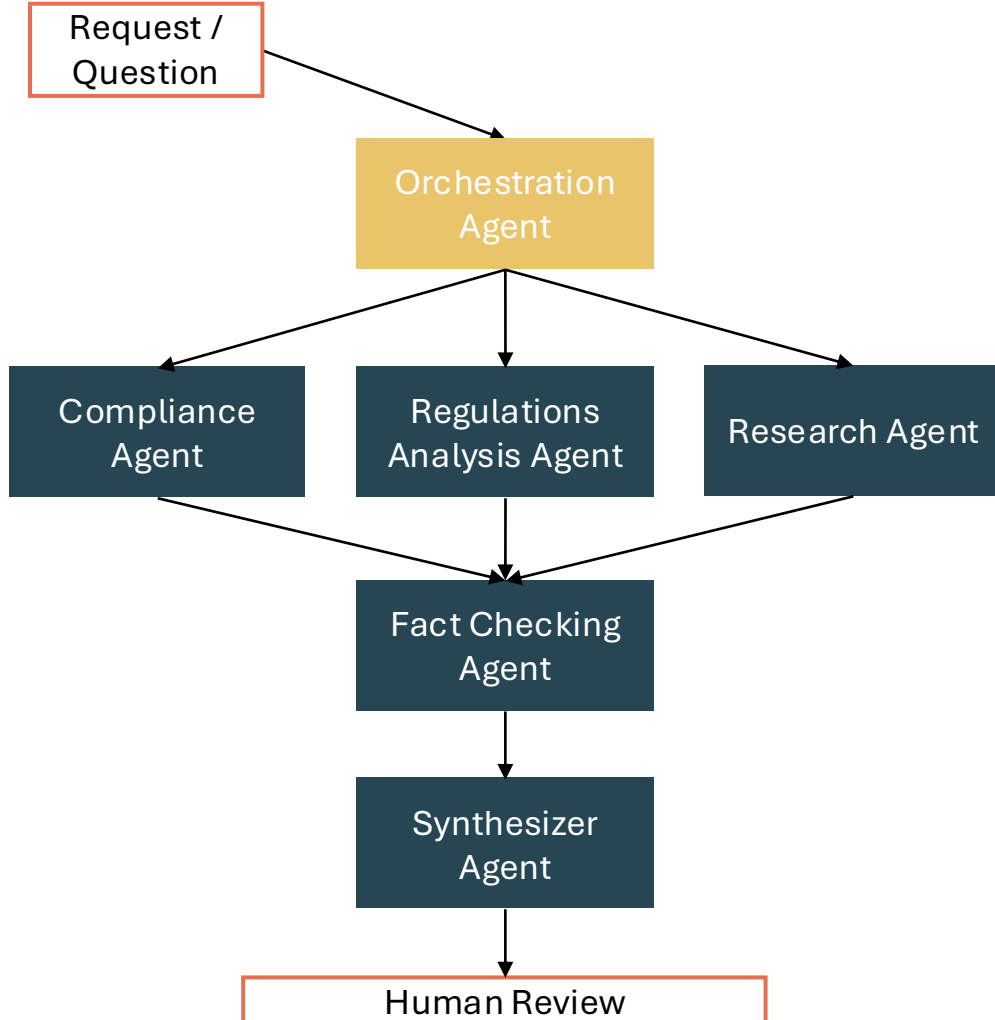
التوجيه الأنسب هو استخدام وكيل منسق Orchestrator يدير دفعة تنفيذ المهام ويفرض قيودا على الوكلاء الفرعيين من ناحية: Max Tokens، Max Tools Calls، Max API Calls، Timeout...

يبني الوكيل المنسق شبكة الاعتماد بين الوكلاء المتخصصين Dependency Graph ويحدد ما يتم تنفيذه على التوالي والتوازي ولديه صلاحية انشاء عدد ديناميكي من الوكلاء الفرعيين بحسب صعوبة المهمة\السؤال.

يمكن استخدام ذاكرة مشتركة Shared Memory بين الوكلاء إن لزم.

بحسب حساسية المشروع، يمكن تطبيق Human in the Loop بحيث تتم مراجعة بشرية للنتائج النهائية.

هل أكتفي بوكيل واحد للبحث في الاقتصاد السياسي وتشريعاته أم ألجأ لنظام متعدد الوكلاء (أي وكيل للبحث، وكيل للتشريعات، وكيل للتحقق من النتائج...)?



متى نستخدم نظام متعدد الوكلاء (تنسق فيما بينها بدون منسق مركزي):

- الحاجة للمعالجة بالتوازي Parallel processing
- وجود أنماط تفكير وتحليل مختلفة
- وجود سير عمل Workflows معقد مع اعتمادية بينية Dependencies

متى نستخدم نظام متعدد الوكلاء بوجود منسق مركزي (كما في الصورة):

- وجود مهام واضحة ومحددة (أغلب المشاريع عمليا)
- رغبة بإظهار الإجابات بسرعة

هل ألجأ للضبط الدقيق للنموذج اللغوي Fine Tuning أم ألجأ لتقنيات RAG؟

من الخطأ التوجه مباشرة للضبط الدقيق عبر LoRA or QoRA، لأنه قد لا يكون الحل الأكفأ.

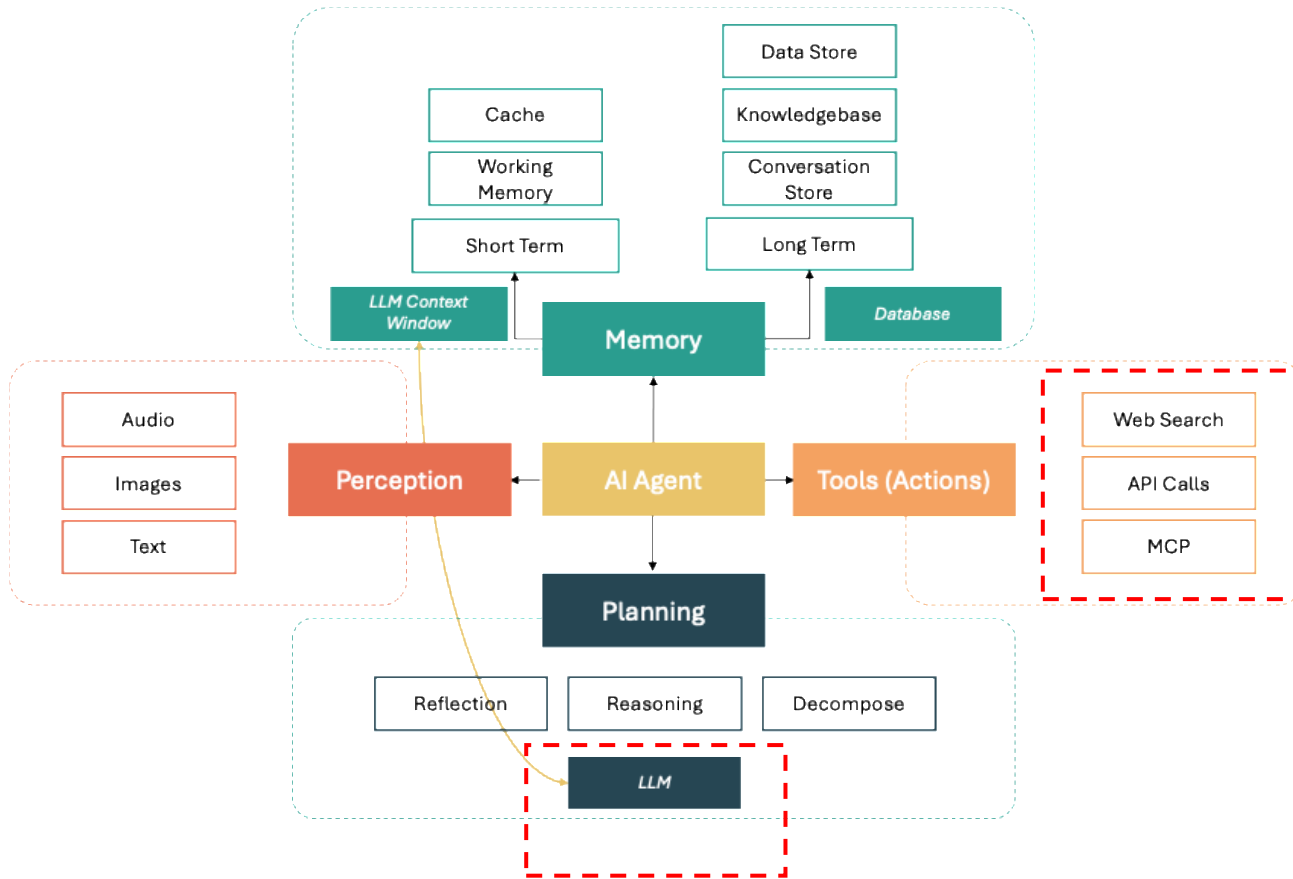
قم باستخدام FT عندما تكون قاعدة المعلومات ثابتة Static، وعند وجود بيانات عالية الجودة و Domain Specific ليتم تدريب النموذج عليها (ألف إلى ٣ آلاف مثال) وعند رغبتك في خفض ال Latency بشكل كبير، وفي حال كان مقبولا استخدام البيانات في التدريب من ناحية ضوابط الامتثال Compliance.

عدا عن ذلك استخدم RAG. يوجد أيضا توجه لنظام هجين RAFT:

Most high-performing systems now use **Retrieval-Augmented Fine-Tuning (RAFT)**.

1. **Fine-tune** a small model (e.g., Llama 3 or Mistral) to understand your industry's terminology and desired output format. [🔗](#)
2. **Layer RAG** on top so that this "expert" model can look up the latest facts. [🔗](#)

Example: A medical AI is **fine-tuned** to understand clinical reasoning and specialized terminology, but it uses **RAG** to look up the specific latest drug interactions from a live database. [🔗](#)

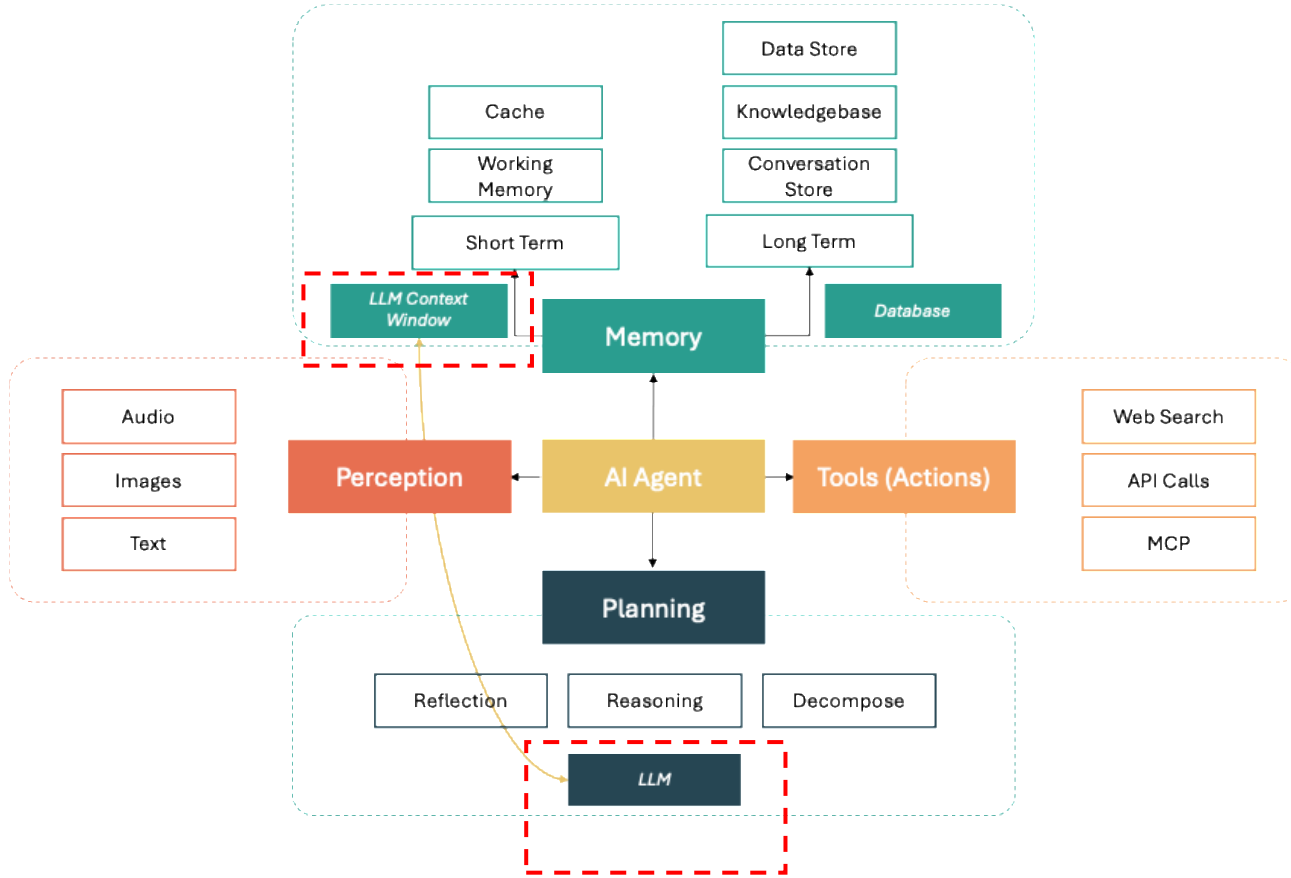


أستخدم LLM لديه Context Window ضخمة تتسع لمليون Token. هل هناك ما يدعو للقلق؟

يمكن للنموذج اللغوي أن يشهد انخفاضا ملحوظا في قدرته على المعالجة والتفكير، وهو ما يعرف بفساد السياق Context Rot أو Context Window Decay.

أهم أعراض هذه الظاهرة:

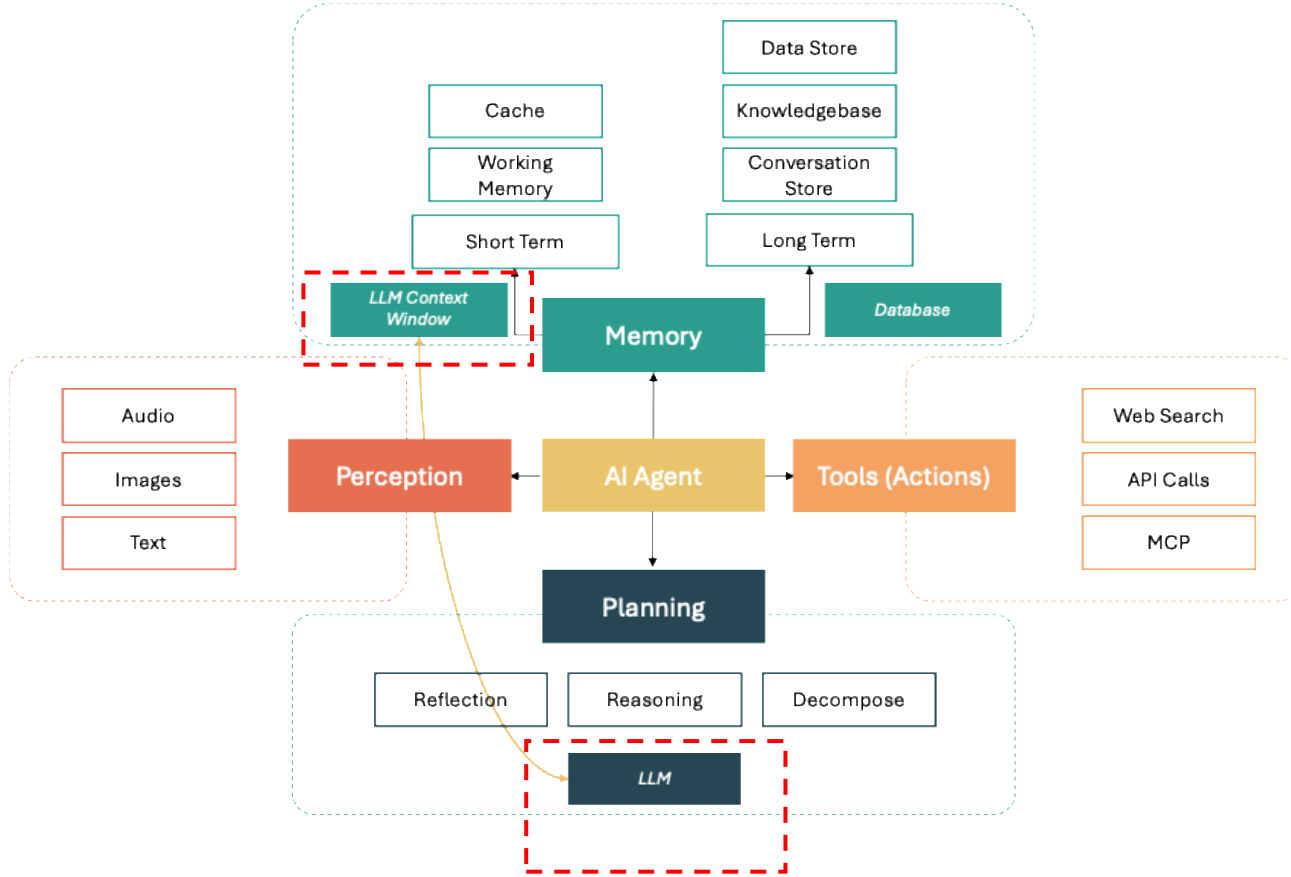
- تجاهل النموذج اللغوي للأوامر Instructions Drift
- الخلط في أسماء الكيانات (أسماء الأماكن، البشر...) Entity Confusion
- إجهاد التفكير والميل لإخراج نتائج سطحية Reasoning Fatigue
- تشتت الانتباه Attention Dilution وحصول مشكلة Lost in the Middle بحيث ينسى النموذج تسلسل المهمة المطلوبة



أستخدم LLM لديه Context Window ضخمة تتسع لمليون Token. هل هناك ما يدعو للقلق؟

علاج هذه الظاهرة يكون عبر:

- التشذيب Pruning عبر إزالة الـ Tokens ذات القيمة المنخفضة (مثل كلمات الشكر) وعدم تمريرها للنموذج
- التلخيص Summarization ولكن مع الحذر من Semantic Compression Loss
- ضغط السياق Context Compaction أي بدلا من وضع محتوى الملفات في نافذة السياق، يتم وضع إشارة للملف Reference ليتم جلب محتواه عند الحاجة
- استخدام استراتيجية أوامر Prompting Strategy فعالة مثل Skeleton of Thought

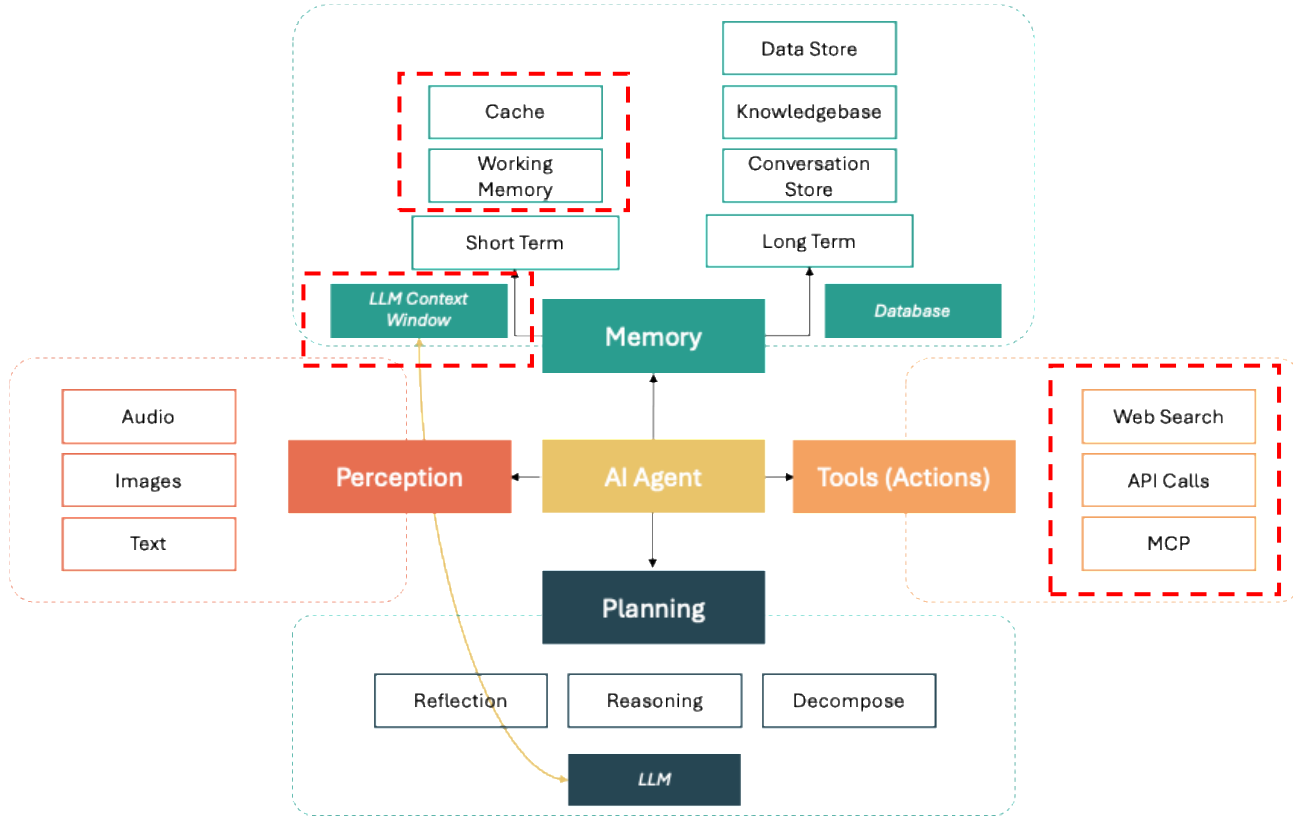


ما هو الفرق بين Context Rot و Context Bloat؟

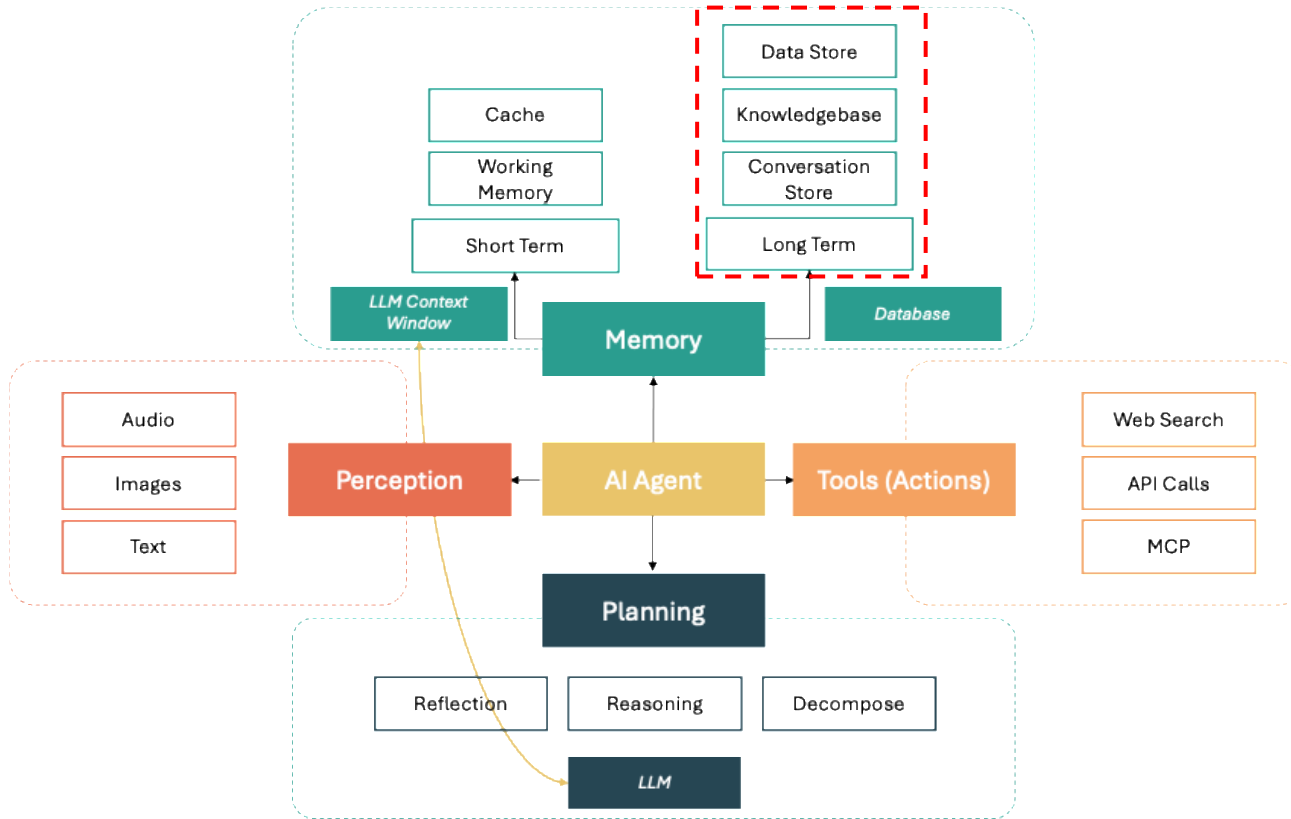
فساد السياق Context Rot هي مشكلة خاصة بالنموذج اللغوي، بينما انتفاخ السياق Context Bloat هي مشكلة في الإدخال، وتصيب الذاكرة العاملة للوكيل Working Memory المستخدمة في تنفيذ كل مهمة.

تحدث هذه المشكلة عندما تنخفض جودة ما يحتويه هذا السياق من معلومات منخفضة القيمة، مبعثرة، غير ذات صلة. وهناك عدة أسباب لحصولها:

- مخرجات الأدوات Tool output junk (مثلا أن تكون القيمة المرتجعة على شكل HTML، أو أن يتم اختيار الأداة الخاطئة للسؤال، أو الفشل الصامت Silent failure of MCP server)
- التفكير التكراري Repetitive reasoning وانحباسه في Loops
- الإغراق بالأوامر (شروحات مطولة للأنظمة، كيفية استخدام عشرات الأدوات...)
- فوضى متراكمة مع تتابع المهام



هل يكون معيار اختياري للـ Vector DB هو سرعة استرجاع النتائج فقط؟



بجانب معيار الـ Latency، هناك معايير إضافية مهمة:

- التوسع Scalability وعدد الـ Vectors
- آلية الـ Indexing وسرعته
- دعمها لضغط المتجهات Vector Compression
- قيمة الـ Operational Overhead

بما يخص دقة استرجاع النتائج من قاعدة بيانات المتجهات يقترح المهندس هشام هارون الصيغة الموزونة التالية:

$$\text{Score} = (\text{Vector score} \times 5) + (\text{BM25} \times 3) + (\text{Recency Score} \times 0.2)$$

ويمكن استخدام المقاييس التالية لقياس سرعة ودقة أداء الاسترجاع:

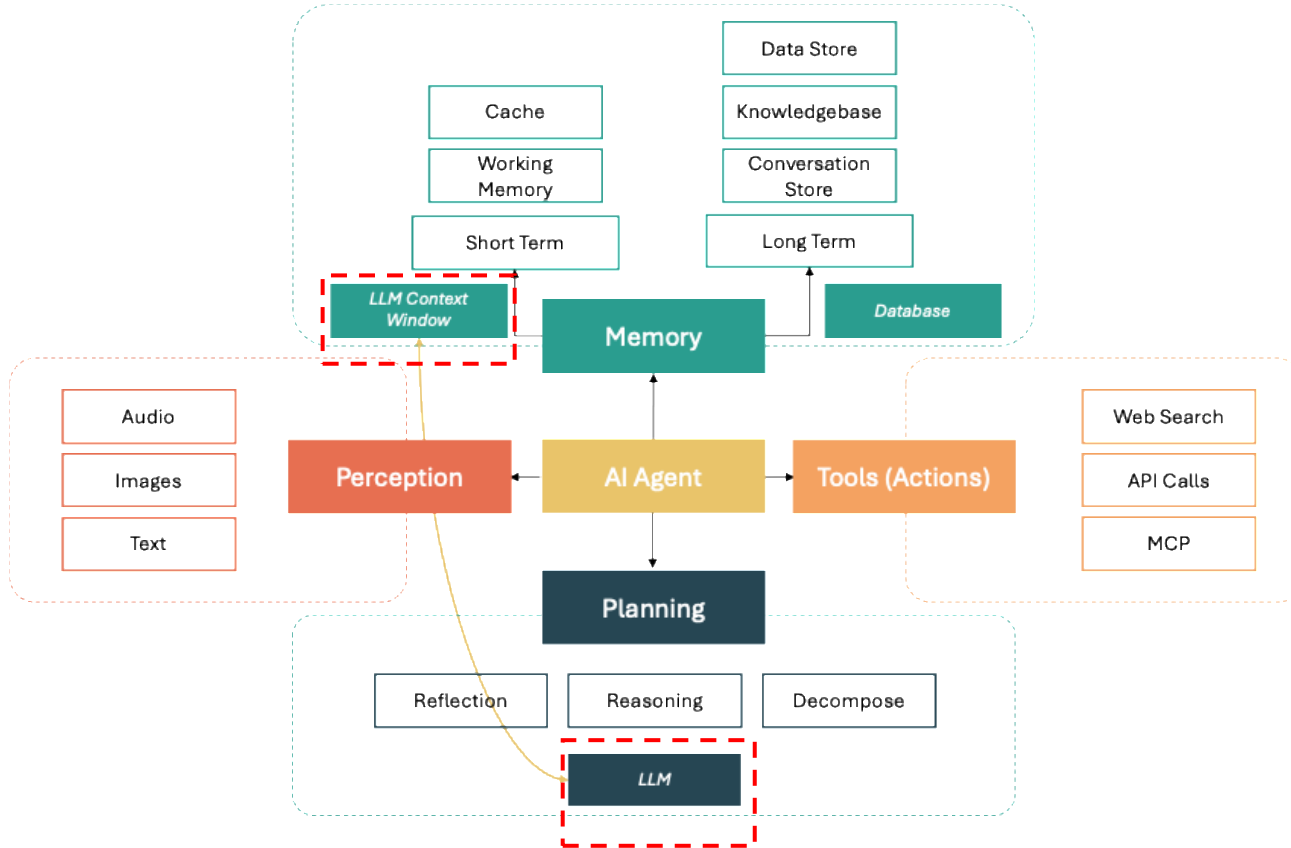
Retrieval quality metrics

Recall @ k
Precision @ k
MRR
NDCG
Hit rate

Latency metrics

P50 (median/typical experience)
P99 (worst case experience)

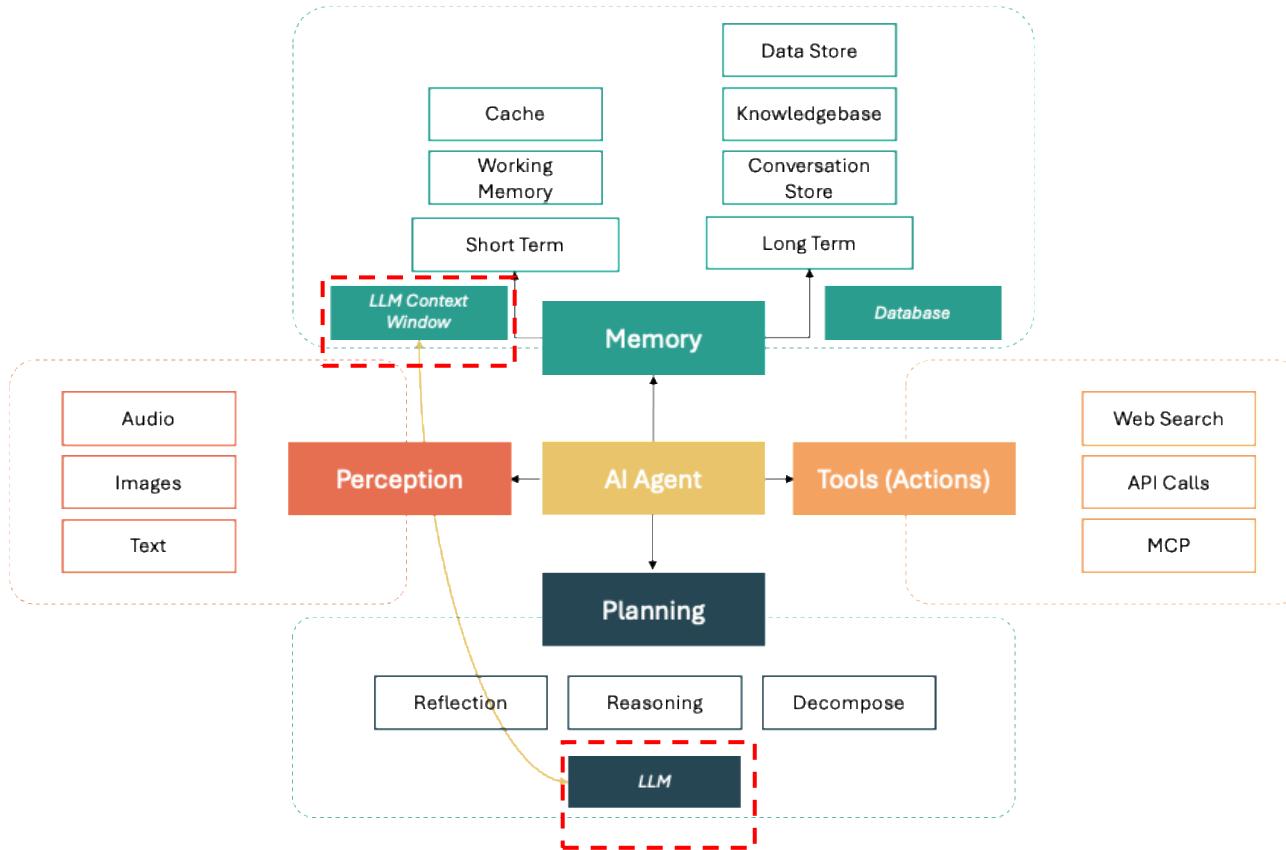
ملاحظات عامة أثناء التطوير ١/٣



- تجنب زيادة إنتاجية النموذج اللغوي على توليد ومعالجة البيانات
Throughput عبر زيادة سعة الـ Batch، وانما استخدم:
Continuous Batching (Iteration-level Batching)
بالإضافة لضغط النماذج (Quantization (INT8, FP8, AWQ)
- بالرغم من أن الـ MCP يقدم حلا مهما في جانب الربط
Integration وحل المشكلة المعروفة بـ $N \times M$ ، من المهم التنبيه
لمشاكله الأمنية من قبيل Prompt Injection
لخفض عدد الـ Tokens المستخدمة وترشيد الـ Token Economics أو Tokenomics، يتوجب مراجعتها بالتفصيل
في:

- System Prompt
- Retrieved docs
- Conversation history
- User query

ملاحظات عامة أثناء التطوير ٢/٣



- راجع استراتيجيات التجزئـة Chunking المناسبة لحالة استخدامك:
 - Basic: word, recursive, token...
 - Semantic: meaning based
 - LLM-based: contextual chunking
- استخدم نماذج Embedding models تحافظ على المعلومات الدلالية في أبعاد أصغر Smaller dimensions، وبالنسبة لتعدد اللغات، استخدم نماذج تتعامل مع ذلك في ذات الفضاء Same Embedding space بدلاً من وجود فضاء مختلف لكل لغة.
- احذر من انهيار فضاء التضمين Embedding Space Collapse، فهو انهيار صامت Silent Failure، يؤدي لحصر المتجهات المولدة في فضاء فرعي محدود Limited Subspace، يجعل من متجهات Apple ~ Banana ~ Car متشابهة جداً

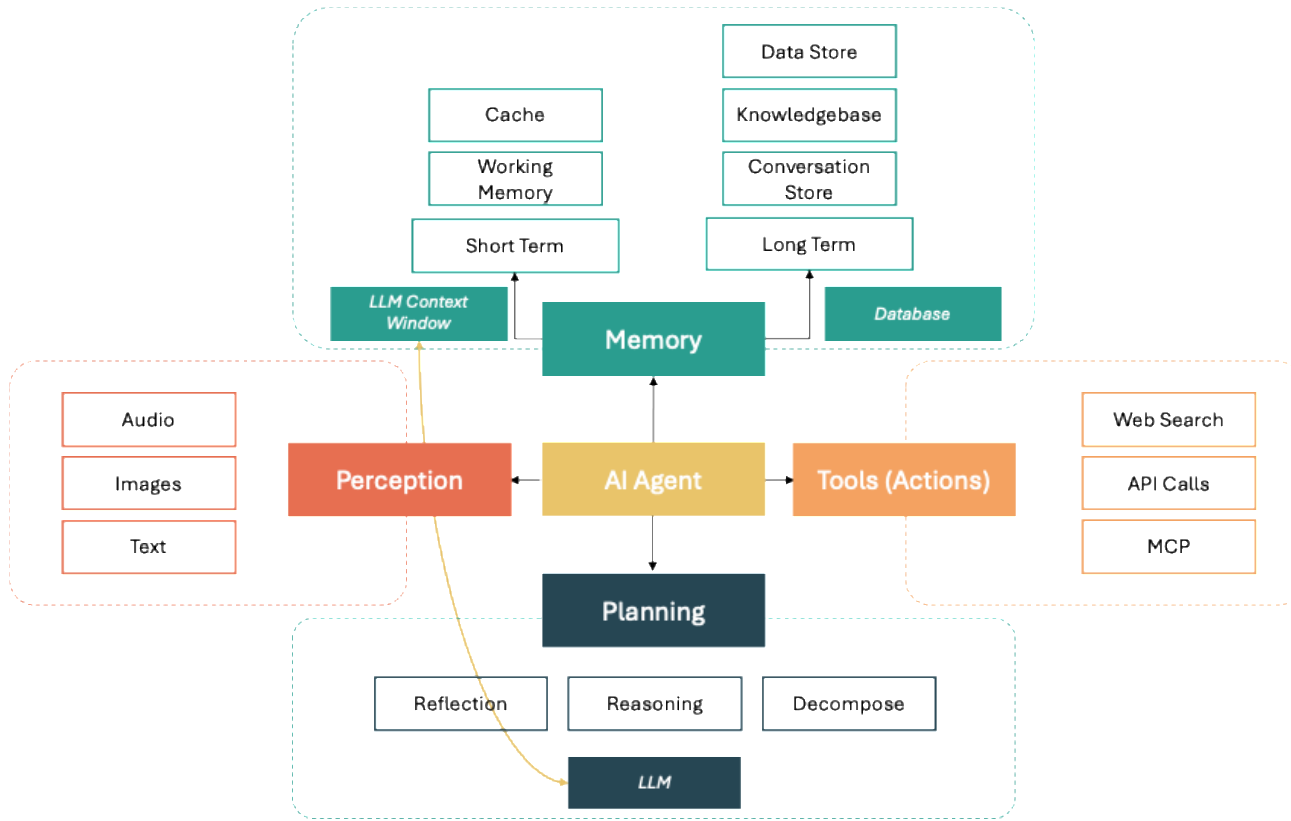
ملاحظات عامة أثناء التطوير ٣/٣

Knowledge Base Size	Performance State	Experience
0 - 5k Docs	The Honeymoon	High accuracy, sub-second response. Standard vector search works.
5k - 15k Docs	The Drift	Occasional "weird" answers. You start needing a Reranker.
20k+ Docs	The Cliff	Sudden drop. Hallucinations spike. Precision drops by ~80%.

$$\text{Score} = (\text{Vector score} \times 5) + (\text{BM25} \times 3) + (\text{Recency Score} \times 0.2)$$

- احذر من Prompt Drift وهو أحد الأعطال الصامتة، ويحدث عند تبدل النتائج المرتجعة بالرغم من إبقاء الـ Prompt على حاله بسبب تغيرات طرأت على النموذج من المزود، أو تغير قواعد البيانات في RAG (مثل زيادة مستوى تعقيدها، أو الـ Format...). ويكون الحل بتفعيل الـ Automated Evaluations.
- احذر من RAG Scaling Cliff وهو تدني جودة الإجابات المرتجعة في RAG وزيادة الوقت بشكل ملحوظ بعد نقل المشروع للـ Production (انظر الصورة)
- لمعالجة هذا التحدي، يمكنك استخدام:
 - آلية البحث الهجينة (التي تم عرضها في السابق <===)
 - إضافة طبقة إعادة ترتيب Reranking
 - استخدام GraphRAG بدلا من Vector RAG
- يمكن عمل FT لنموذج صغير بهدف تحديد أفضل أداة لاستدعائها.
- determine best tool calling.
- البيانات الوصفية للمستندات Docs Metadata يمكنها تعزيز جودة استرجاع البيانات بجانب البحث المتجهي Vector Search.

ما هي التوجيهات العامة لتهيئة المشروع لمرحلة Production؟



- وضع قيود صارمة على الميزانية المخصصة لـ Tokens، Tool Calls، API Calls، Timeout...
- وضع حواجز أمان Guardrails لموضوع الخصوصية، الأمان، وانحيازات النماذج
- المراقبة الآنية المستمرة Monitoring لـ Cost, Latency, error rate
- تسجيل وفحص كل قرار Audit logging for each decision
- التعامل مع الأخطاء وآلية تراجع Error handling and rollback mechanism
- التوثيق والامتثال Compliance and Documentation
- التقييم والقياس Evals